

Aula 04 — Arquiteturas (parte 2)

Resumo 5 de 14 · arquivo aula04 · 18 imagens / 14 slides · Cap. 2 de van Steen & Tanenbaum

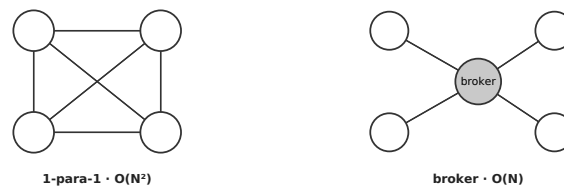
Continuação de arquiteturas, com dois focos: **middleware** (como organizá-lo — wrappers, brokers, interceptadores, adaptação) e as **arquiteturas centralizadas cliente-servidor** (modelos de 1, 2 e 3 camadas), fechando com exemplos reais (NFS e servidores web).

1. Middleware: o “SO” dos sistemas distribuídos

O middleware faz para um sistema distribuído o que o sistema operacional faz para uma máquina: é um **gerenciador de recursos** que cuida de comunicação entre aplicações, segurança e mascaramento/recuperação de falhas, reunindo funções comuns para que cada aplicação não precise reimplementá-las.

Wrappers (adaptadores)

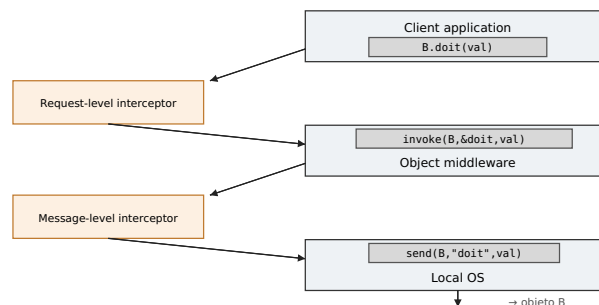
Muitas vezes montamos um sistema juntando **componentes legados**. O problema é que a interface do componente antigo raramente serve a todas as aplicações. A solução é um **wrapper** (ou adaptador): uma “casca” que oferece uma interface compreensível para o cliente e traduz para as funções do componente (ex.: uma casca RESTful sobre um sistema antigo). Ao ligar N componentes, a forma de organizar os wrappers muda drasticamente a complexidade:



Ligando N componentes: interligar todos “1-para-1” exige $N \times (N-1) = O(N^2)$ wrappers; usar um broker central exige só $2N = O(N)$.

Interceptadores

Um **interceptador** é um trecho de software inserido no fluxo de execução para **alterar o comportamento** sem modificar diretamente o middleware — útil, por exemplo, para acrescentar funcionalidade quando o objeto de destino está replicado ou o dado é grande. Uma chamada aparentemente local (`B.doit(val)`) atravessa vários níveis, e em cada nível pode ser interceptada:



A chamada desce da aplicação → middleware de objetos → SO local, e em dois pontos pode ser desviada por interceptadores (nível de requisição e nível de mensagem).

Middleware adaptável

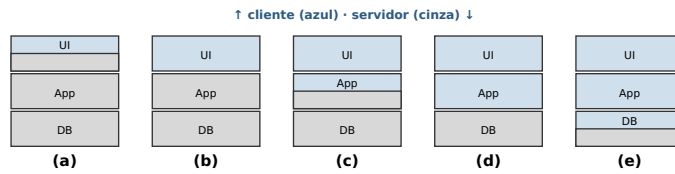
Precisa se ajustar a **mudanças no ambiente** (mobilidade, falhas de rede/hardware) e **em tempo de execução** (sistemas que não podem ser desligados). Técnicas: **configuração dinâmica**, *late binding* e *dynamic loading*. Os desafios: aumenta a responsabilidade do middleware, é difícil manter o estado coerente e as dependências entre componentes ficam complexas.

2. Arquiteturas centralizadas (cliente-servidor)

No **modelo cliente-servidor** básico há processos que **oferecem** serviços (servidores) e processos que os **usam** (clientes), possivelmente em máquinas diferentes, comunicando-se por **requisição/resposta** (sobre UDP ou TCP). Ressalva importante: a distinção entre cliente e servidor *nem sempre é óbvia* — um servidor pode ser cliente de outro.

Organizações tradicionais (1, 2 e 3 camadas)

Uma camada: terminal burro/mainframe. Duas camadas: cliente/servidor único. Três camadas: cada camada numa máquina separada. As **configurações de duas camadas** variam conforme quanto da pilha (interface, aplicação, dados) fica no cliente ou no servidor:



Da esquerda (cliente "magro") para a direita (cliente "gordo"): cada vez mais da pilha migra para o cliente.

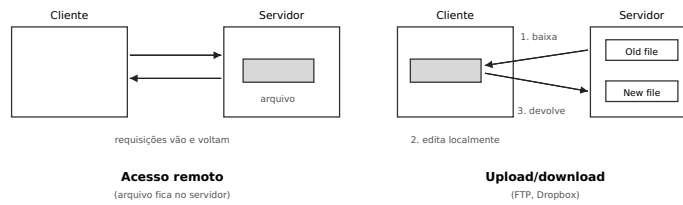
Cinco configurações de duas camadas: em (a) só parte da interface fica no cliente; em (e) o cliente tem interface, aplicação e até parte do banco de dados.

Quando cada camada fica numa máquina diferente, temos a **arquitetura de três camadas**: o cliente fala com um **servidor de aplicação (AS)**, que por sua vez fala com um **servidor de banco de dados (DS)**. Aqui o servidor de aplicação é, ao mesmo tempo, servidor (do cliente) e cliente (do BD) — um exemplo é o TPM (Transaction Processing Monitor).

3. Exemplos reais

Sistemas de arquivos em rede (NFS)

Cada servidor **NFS** oferece uma **visão padronizada** do seu sistema de arquivos local — o mesmo modelo, independentemente da implementação real. Há dois modelos de acesso:



Dois modelos de acesso a arquivos: **acesso remoto** (o arquivo permanece no servidor e cada operação é uma requisição) × **upload/download** (baixa o arquivo, edita local e devolve — modelo do FTP/Dropbox).

Servidores web

“Nos velhos tempos” um site era só uma **coleção de arquivos HTML** ligados por hiperlinks: o servidor achava o arquivo pelo link e o navegador o renderizava. Depois, com sites construídos em torno de um **banco de dados**, passou a existir um programa separado — a **CGI (Common Gateway Interface)** — que **compõe a página dinamicamente** a partir do conteúdo, antes de o navegador renderizá-la.